

Strings Solved Questions

All 38 problems from Part 14 of the Strings guide, worked in full — the problem, the approach, the complexity, the tested C++ solution, and the mistake that most often breaks it.

38

PROBLEMS

14

PATTERNS

216

TESTS PASSING

3

TIERS

Prepared for **Hari** · on-campus placement preparation

Every solution below is extracted verbatim from a file that compiles clean and passes 216 tests

Generated 15 September 2026

Contents

Tier 1 • Fundamentals

01	LC 344 · Reverse String
02	LC 125 · Valid Palindrome
03	LC 242 · Valid Anagram
04	LC 387 · First Unique Character in a String
05	LC 383 · Ransom Note
06	LC 14 · Longest Common Prefix
07	LC 58 · Length of Last Word
08	LC 28 · Find the Index of the First Occurrence in a String
09	LC 205 · Isomorphic Strings
10	LC 392 · Is Subsequence
11	LC 13 · Roman to Integer
12	LC 12 · Integer to Roman
13	LC 415 · Add Strings

Tier 2 • The core patterns

14	LC 3 · Longest Substring Without Repeating Characters
15	LC 438 · Find All Anagrams in a String
16	LC 567 · Permutation in String
17	LC 424 · Longest Repeating Character Replacement
18	LC 76 · Minimum Window Substring
19	LC 49 · Group Anagrams
20	LC 5 · Longest Palindromic Substring
21	LC 647 · Palindromic Substrings
22	LC 680 · Valid Palindrome II
23	LC 151 · Reverse Words in a String
24	LC 443 · String Compression
25	LC 8 · String to Integer (atoi)
26	LC 165 · Compare Version Numbers
27	LC 6 · Zigzag Conversion
28	LC 179 · Largest Number

Tier 3 • Advanced

29	LC 214 · Shortest Palindrome
30	LC 459 · Repeated Substring Pattern
31	LC 1392 · Longest Happy Prefix

- 32 LC 187 · Repeated DNA Sequences
- 33 LC 516 · Longest Palindromic Subsequence
- 34 LC 1143 · Longest Common Subsequence
- 35 LC 72 · Edit Distance
- 36 LC 10 · Regular Expression Matching
- 37 LC 44 · Wildcard Matching
- 38 LC 97 · Interleaving String

Tier 1 · Fundamentals

Characters, counting and the basic two-pointer shapes. Do all thirteen of these before touching Tier 2 — everything later assumes this is automatic.

01 LC 344 · Reverse String

EASY Two pointers, converging

PROBLEM

Reverse a `vector<char>` in place. You must do it with $O(1)$ extra memory — no second array, no `reverse` call.

APPROACH

One index at each end, walking towards each other, swapping as they go.

The loop condition is `l < r`, not `l <= r`. At `l == r` you are looking at a single middle character, which is already in the right place; swapping it with itself is harmless but pointless, and `<` handles odd and even lengths with no special case at all.

Complexity $O(n)$ time — each character is touched once. $O(1)$ extra space.

C++ SOLUTION

```
class Solution {
public:
    void reverseString(vector<char>& s) {
        int l = 0, r = (int)s.size() - 1;
        while (l < r) {
            swap(s[l], s[r]);
            l++;
            r--;
        }
    }
};
```

WATCH OUT

Writing `while (l <= r)` is not wrong here, but the same habit breaks in problems where the middle element must be treated differently. Learn `l < r` as the default and deviate deliberately.

02 LC 125 · Valid Palindrome

EASY

Two pointers with filtering

PROBLEM

A phrase is a palindrome if, after keeping only alphanumeric characters and lower-casing them, it reads the same forwards and backwards. Decide whether the given string qualifies.

APPROACH

Do not build a cleaned copy — that is an extra $O(n)$ allocation you do not need. Walk two pointers inwards and skip the characters you do not care about as you go.

Inside the main loop, advance `l` past non-alphanumerics and retreat `r` past them, then compare the two survivors case-insensitively.

Complexity $O(n)$ time — each index is visited at most once by either pointer. $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool isPalindrome(string s) {
        int l = 0, r = (int)s.size() - 1;
        while (l < r) {
            // both guards repeat `l < r`, or a pure-punctuation string walks off
            while (l < r && !isalnum((unsigned char)s[l])) l++;
            while (l < r && !isalnum((unsigned char)s[r])) r--;
            if (tolower((unsigned char)s[l]) != tolower((unsigned char)s[r]))
                return false;
            l++;
            r--;
        }
        return true;
    }
};
```

WATCH OUT

Both inner skip loops need `l < r` repeated in their conditions. Without it, an input like `".,.,."` sends `l` walking straight off the end of the string and you read out of bounds. This is the single most common failure on this problem.

03 LC 242 · Valid Anagram

EASY Frequency counting

PROBLEM

Return true if `t` is a rearrangement of `s`. Both consist of lowercase English letters.

APPROACH

Guard the lengths first — different lengths can never be anagrams, and the guard is what makes the rest of the argument work.

Then count up from `s` into a 26-slot array and count down from `t`. If any count goes negative, `t` has more of that letter than `s` had, so bail out immediately. If nothing goes negative and the sizes were equal, nothing can be left over either, so the answer is true.

Complexity $O(n)$ time, $O(26) = O(1)$ space. Sorting both strings also works but is $O(n \log n)$.

C++ SOLUTION

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) return false;
        int cnt[26] = {0}; // {0} zeroes all 26 slots
        for (char c : s) cnt[c - 'a']++;
        for (char c : t)
            if (--cnt[c - 'a'] < 0) return false; // t has more of c than s did
        return true; // equal sizes => nothing left over
    }
};
```

WATCH OUT

`int cnt[26];` without `= {0}` is filled with garbage. The code compiles, runs, and is wrong differently on every machine. Write `int cnt[26] = {0};` every single time — the `{0}` zeroes all 26 slots, not just the first.

04 LC 387 · First Unique Character in a String

EASY Frequency counting, two passes

PROBLEM

Return the index of the first character in `s` that appears exactly once, or `-1` if there is none.

APPROACH

Two passes. The first fills a 26-slot count array. The second walks the string **in order** and returns the index of the first character whose count is 1.

The second pass has to be over the string, not over the count array — the count array has lost all position information, and the question asks for the *first* such character.

Complexity $O(n)$ time (two linear passes), $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int firstUniqChar(string s) {
        int cnt[26] = {0};
        for (char c : s) cnt[c - 'a']++;
        for (int i = 0; i < (int)s.size(); i++)
            if (cnt[s[i] - 'a'] == 1) return i;
        return -1;
    }
};
```

WATCH OUT

Scanning the 26-slot array instead of the string finds the alphabetically-smallest unique character, not the first one. On `"loveleetcode"` that returns the wrong index.

05 LC 383 · Ransom Note

EASY Frequency counting

PROBLEM

Can `ransomNote` be built using only the letters available in `magazine`, with each letter used at most as many times as it appears there?

APPROACH

Exactly the second half of Valid Anagram, with the length guard removed. Count the magazine's letters up, then count the note's letters down, and fail the moment a count goes negative.

Note the direction: you count the **supply** up and the **demand** down. Getting them the wrong way round answers a different question.

Complexity $O(n + m)$ time, $O(26) = O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        int cnt[26] = {0};
        for (char c : magazine) cnt[c - 'a']++;
        for (char c : ransomNote)
            if (--cnt[c - 'a'] < 0) return false;
        return true;
    }
};
```

WATCH OUT

There is no size guard here, and there should not be — a short note from a long magazine is perfectly valid. Copying the guard across from LC 242 rejects every legitimate case.

06 LC 14 · Longest Common Prefix

EASY Vertical scan

PROBLEM

Find the longest string that is a prefix of every string in the array. Return "" if there is none.

APPROACH

Scan **column by column** rather than string by string: take the i -th character of the first word and check it against the i -th character of every other word. The moment one word is too short or disagrees, the answer is everything before column i .

This stops as early as possible — the work is bounded by the length of the answer times the number of words, not by the total input.

Complexity $O(S)$ where S is the sum of all characters, and much better in practice. $O(1)$ extra space.

C++ SOLUTION

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        if (strs.empty()) return "";
        // vertical scan: column by column across every word
        for (int i = 0; i < (int)strs[0].size(); i++) {
            char c = strs[0][i];
            for (int j = 1; j < (int)strs.size(); j++)
                if (i >= (int)strs[j].size() || strs[j][i] != c)
                    return strs[0].substr(0, i);
        }
        return strs[0];
    }
};
```

WATCH OUT

`strs[j][i]` when word j is shorter than i is undefined behaviour. The length check `i >= strs[j].size()` must come **first** in the `||`, because `||` short-circuits left to right.

07 LC 58 · Length of Last Word

EASY

Reverse scan

PROBLEM

Return the length of the last word in a string, where a word is a maximal run of non-space characters. The string may have trailing spaces.

APPROACH

Walk backwards from the end. First skip every trailing space, then count characters until you hit a space or run out of string.

Two loops, in that order. Scanning forwards and remembering the last word also works but touches every character; the backward scan touches only the trailing spaces plus the last word.

Complexity $O(n)$ worst case, $O(\text{length of last word} + \text{trailing spaces})$ in practice. $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int lengthOfLastWord(string s) {
        int i = (int)s.size() - 1;
        while (i >= 0 && s[i] == ' ') i--;    // skip trailing spaces first
        int len = 0;
        while (i >= 0 && s[i] != ' ') { len++; i--; }
        return len;
    }
};
```

WATCH OUT

Skipping the trailing spaces is a separate loop that must run *first*. Starting the count immediately on an input like "day " returns 0.

08 LC 28 · Find the Index of the First Occurrence in a String

EASY Naive pattern matching

PROBLEM

Return the index of the first occurrence of `needle` in `haystack`, or `-1`. By convention an empty needle matches at index 0.

APPROACH

The naive scan: for every possible start position, compare characters until they disagree or the needle is exhausted.

The loop bound is `i + m <= n`, which stops before the needle could run past the end of the haystack. That single expression removes every bounds check from the inner loop.

KMP (problem 29) gets this to $O(n + m)$, and is worth mentioning — but for a single search on ordinary text the naive version is what an interviewer expects first.

Complexity $O(n \cdot m)$ worst case, close to $O(n)$ on ordinary text. $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int strStr(string haystack, string needle) {
        int n = haystack.size(), m = needle.size();
        if (m == 0) return 0;
        for (int i = 0; i + m <= n; i++) {           // stop before running off the end
            int j = 0;
            while (j < m && haystack[i + j] == needle[j]) j++;
            if (j == m) return i;
        }
        return -1;
    }
};
```

WATCH OUT

The empty-needle case must return 0, not `-1`. And writing the loop as `i < n` rather than `i + m <= n` means the inner comparison reads past the end of the haystack on the last few positions.

09 LC 205 · Isomorphic Strings

EASY Character mapping, both directions

PROBLEM

Two strings are isomorphic if the characters of `s` can be replaced to get `t`, where every occurrence of a character maps to the same character, and **no two characters map to the same one**.

APPROACH

Keep **two** maps, `s → t` and `t → s`, and require both to agree at every position.

The second map is what enforces the no-two-characters-collide rule. With only `s → t`, the pair `("ab", "aa")` passes: `a → a` and `b → a` are individually consistent, but they collapse two characters into one, which the problem forbids.

Complexity $O(n)$ time, $O(128) = O(1)$ space with fixed arrays.

C++ SOLUTION

```
class Solution {
public:
    bool isIsomorphic(string s, string t) {
        if (s.size() != t.size()) return false;
        int mapST[128], mapTS[128];
        memset(mapST, -1, sizeof(mapST));
        memset(mapTS, -1, sizeof(mapTS));
        for (int i = 0; i < (int)s.size(); i++) {
            unsigned char a = s[i], b = t[i];
            // one map is not enough: "ab" -> "aa" passes s->t but fails t->s
            if (mapST[a] == -1 && mapTS[b] == -1) {
                mapST[a] = b;
                mapTS[b] = a;
            } else if (mapST[a] != (int)b || mapTS[b] != (int)a) {
                return false;
            }
        }
        return true;
    }
};
```

WATCH OUT

One map is not enough, and `("ab", "aa")` is the counterexample that proves it. If you can only remember one thing about this problem, remember that it needs the mapping checked in both directions.

10 LC 392 · Is Subsequence

EASY

One pointer per string

PROBLEM

Return true if `s` is a subsequence of `t` — that is, `s` can be obtained from `t` by deleting some characters without changing the order of the rest.

APPROACH

One pointer into `s`, one scan across `t`. Whenever the current character of `t` matches the character `s` is waiting for, advance `s`'s pointer. At the end, `s` is a subsequence exactly when its pointer reached the end.

Greedy is correct, and the exchange argument is worth saying out loud: matching the **earliest** possible position in `t` is never worse, because any later match leaves strictly fewer characters of `t` available for the rest of `s`.

Complexity $O(n + m)$ time, $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool isSubsequence(string s, string t) {
        int i = 0, n = s.size();
        for (char c : t)
            if (i < n && s[i] == c) i++; // greedy: match the earliest position
        return i == n;
    }
};
```

WATCH OUT

The follow-up — many `s` values against one fixed `t` — changes the answer completely: precompute, for each position of `t` and each letter, the next position of that letter, then each query is $O(|s|)$. Interviewers ask this follow-up often.

11 LC 13 · Roman to Integer

EASY Parsing with a look-ahead rule

PROBLEM

Convert a Roman numeral in the range 1–3999 to an integer.

APPROACH

Roman numerals are additive except for six subtractive pairs. Rather than special-casing those six, use the rule that generates them: **a symbol whose value is smaller than the one immediately after it is subtracted**, and every other symbol is added.

So walk left to right, look one character ahead, and choose the sign. No lookup of pairs needed.

Complexity $O(n)$ time, $O(1)$ space (the value table has seven fixed entries).

C++ SOLUTION

```
class Solution {
public:
    int romanToInt(string s) {
        unordered_map<char, int> v{{'I', 1}, {'V', 5}, {'X', 10}, {'L', 50},
                                   {'C', 100}, {'D', 500}, {'M', 1000}};

        int n = s.size(), res = 0;
        for (int i = 0; i < n; i++) {
            // a smaller value immediately before a larger one is subtracted
            if (i + 1 < n && v[s[i]] < v[s[i + 1]]) res -= v[s[i]];
            else res += v[s[i]];
        }
        return res;
    }
};
```

WATCH OUT

The look-ahead needs the bound `i + 1 < n`, or the last character reads past the end. And the comparison is strictly `<` — equal adjacent symbols like `"II"` must add, not subtract.

12 LC 12 · Integer to Roman

MEDIUM Greedy over a table

PROBLEM

Convert an integer in the range 1–3999 to its Roman numeral.

APPROACH

Put the six subtractive forms — 900/CM, 400/CD, 90/XC, 40/XL, 9/IX, 4/IV — **into the value table** alongside the seven plain symbols, sorted descending. Then a plain greedy loop works: repeatedly subtract the largest value that fits and append its symbol.

Building the table that way is the entire trick. With it, there are no special cases; without it, you need six of them.

Complexity $O(1)$ time — at most 13 table entries and a bounded number of repeats each. $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    string intToRoman(int num) {
        // the six subtractive forms live in the table, so there are no special cases
        const int val[] = {1000,900,500,400,100,90,50,40,10,9,5,4,1};
        const string sym[] = {"M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"};
        string res;
        for (int i = 0; i < 13; i++)
            while (num >= val[i]) { num -= val[i]; res += sym[i]; }
        return res;
    }
};
```

WATCH OUT

Greedy is only correct because the table includes the subtractive forms. Drop them and 4 renders as "IIII", 9 as "VIIII". The table construction *is* the proof of correctness.

13 LC 415 · Add Strings

EASY Digit arithmetic from the end

PROBLEM

Add two non-negative integers given as decimal strings, without converting them to an integer type and without using a big-integer library.

APPROACH

Exactly how you add by hand: one index at the end of each string, walk left, add the two digits plus the carry, append the units digit of the sum and keep the tens as the new carry.

Append forwards and `reverse` once at the end. Inserting at the front instead shifts the whole string on every digit and turns an $O(n)$ routine into $O(n^2)$.

Complexity $O(\max(n, m))$ time, $O(\max(n, m))$ space for the result.

C++ SOLUTION

```
class Solution {
public:
    string addStrings(string num1, string num2) {
        int i = (int)num1.size() - 1, j = (int)num2.size() - 1, carry = 0;
        string res;
        while (i >= 0 || j >= 0 || carry) { // `|| carry` handles "99" + "1"
            int a = i >= 0 ? num1[i--] - '0' : 0;
            int b = j >= 0 ? num2[j--] - '0' : 0;
            int s = a + b + carry;
            res += char('0' + s % 10);
            carry = s / 10;
        }
        reverse(res.begin(), res.end()); // build forwards, reverse once
        return res;
    }
};
```

WATCH OUT

The loop condition needs `|| carry` as a third clause. Without it, `"99" + "1"` produces `"00"` — both strings are exhausted while a carry is still outstanding, and it is silently dropped.

Tier 2 · The core patterns

The tier that actually gets asked. LC 3, LC 5, LC 49, LC 76 and LC 438 are the five most-seen string questions in on-campus drives; be able to write all five from a blank page.

14 LC 3 · Longest Substring Without Repeating Characters

MEDIUM Variable window, longest

PROBLEM

Find the length of the longest substring of `s` containing no repeated character.

APPROACH

The universal window template. A character enters at `r` and increments its count. The window is illegal exactly when that character's count reaches 2 — and checking only the arriving character is enough, because the window was valid before it arrived. Shrink from the left until it is valid again, then record the length.

Because this is a **longest** problem, the recording happens *after* the shrink loop, where the window is guaranteed valid.

The nested `while` is not a second $O(n)$ factor: `l` only ever increases, and it can increase at most n times over the whole run.

Complexity $O(n)$ time — at most $2n$ pointer moves. $O(\min(n, \Sigma))$ space.

C++ SOLUTION

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int cnt[128] = {0}, l = 0, best = 0;
        for (int r = 0; r < (int)s.size(); r++) {
            cnt[(unsigned char)s[r]]++;
            // the arriving character is the only possible offender
            while (cnt[(unsigned char)s[r]] > 1) {
                cnt[(unsigned char)s[l]]--;
                l++;
            }
            best = max(best, r - l + 1);    // record AFTER shrinking: longest
        }
        return best;
    }
};
```

WATCH OUT

"dvdf" is the case that catches the jump variant. If you move `l` straight to `last[s[r]] + 1` without a `max`, an old occurrence drags `l` backwards and the answer comes out as 2 instead of 3.

15 LC 438 · Find All Anagrams in a String

MEDIUM Fixed window + count table

PROBLEM

Return the start indices of every substring of `s` that is an anagram of `p`.

APPROACH

A fixed-size window of width `|p|`. There is no shrink loop at all — one character enters and one leaves on every step.

Maintain a 26-slot count of the window and compare it against the count of `p`. `memcmp` on 26 ints is a handful of instructions, so the comparison is genuinely constant time.

The three index facts, and they are the same three in every fixed window: the character leaving is `s[r - k]`, the window is full from `r == k - 1` onwards, and the window's left end is `r - k + 1`.

Complexity $O(n \cdot 26) = O(n)$ time, $O(26) = O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        int n = s.size(), k = p.size();
        vector<int> res;
        if (n < k) return res;
        int need[26] = {0}, win[26] = {0};
        for (char c : p) need[c - 'a']++;
        for (int r = 0; r < n; r++) {
            win[s[r] - 'a']++;
            if (r >= k) win[s[r - k] - 'a']--;           // the character leaving
            if (r >= k - 1 && memcmp(need, win, sizeof(need)) == 0)
                res.push_back(r - k + 1);               // window's left end
        }
        return res;
    }
};
```

WATCH OUT

Guard `n < k` before you start, or the very first `r >= k - 1` check reads a window that was never filled. And the character leaving is `s[r - k]` — off by one there gives answers that are right on some inputs and wrong on others.

16 LC 567 · Permutation in String

MEDIUM Fixed window + count table

PROBLEM

Return true if `s2` contains any permutation of `s1` as a substring.

APPROACH

Identical to LC 438 in every respect except the answer: instead of collecting the start indices, return true at the first match.

Recognising that these two problems are the same code is worth more than solving either one. If an interviewer asks both, say so.

Complexity $O(n \cdot 26) = O(n)$ time, $O(26) = O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        int n = s2.size(), k = s1.size();
        if (n < k) return false;
        int need[26] = {0}, win[26] = {0};
        for (char c : s1) need[c - 'a']++;
        for (int r = 0; r < n; r++) {
            win[s2[r] - 'a']++;
            if (r >= k) win[s2[r - k] - 'a']--;
            if (r >= k - 1 && memcmp(need, win, sizeof(need)) == 0) return true;
        }
        return false;
    }
};
```

WATCH OUT

"Permutation" and "anagram" mean the same thing here. Some candidates reach for a permutation-generating routine, which is $O(k!)$ and will not finish. The question is about multiset equality, nothing more.

17 LC 424 · Longest Repeating Character Replacement

MEDIUM Window with $\text{len} - \text{maxFreq} \leq k$

PROBLEM

You may change at most k characters of s to any other uppercase letter. Find the length of the longest substring that can be made to contain a single repeated letter.

APPROACH

The validity test is the whole problem: a window of length L whose most frequent character occurs maxFreq times needs exactly $L - \text{maxFreq}$ replacements. So the window is legal while $(r - l + 1) - \text{maxFreq} \leq k$.

maxFreq is deliberately **never decreased** when the window shrinks, so it can be stale — larger than the true maximum of the current window. This is still correct, and the argument is worth rehearsing: a stale maxFreq only makes the test more permissive, so the window never shrinks when it should not; and best can only be updated by a window whose maxFreq was genuinely achieved, because best grows only when the window does.

Complexity $O(n)$ time, $O(26) = O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        // LC 424's alphabet is UPPERCASE, so index by the raw character:
        // a 128-slot table is case-agnostic and costs nothing
        int cnt[128] = {0}, l = 0, maxFreq = 0, best = 0;
        for (int r = 0; r < (int)s.size(); r++) {
            maxFreq = max(maxFreq, ++cnt[(unsigned char)s[r]]);
            // a window of length L needs L - maxFreq replacements
            while ((r - l + 1) - maxFreq > k) {
                cnt[(unsigned char)s[l]]--;
                l++;
            }
            // maxFreq is deliberately never decreased: a stale value only makes
            // the test more permissive, and best is only ever set by a window
            // whose maxFreq was genuinely achieved
            best = max(best, r - l + 1);
        }
        return best;
    }
};
```

WATCH OUT

The alphabet here is **uppercase**, so $c - 'a'$ indexes out of bounds with a negative subscript. Either use $c - 'A'$ or index a 128-slot table by the raw character, which is case-agnostic and costs nothing.

18 LC 76 · Minimum Window Substring

HARD Shrink-while-valid window

PROBLEM

Find the shortest substring of `s` containing every character of `t`, including duplicates. Return `""` if none exists.

APPROACH

This is the template with the `while` condition flipped: shrink **while the window is valid**, recording the answer inside that loop, because that is where the shortest windows live.

Read `need[c]` as a signed debt. Positive means "still owed", zero or negative means "satisfied, with surplus". Testing `need[c]-- > 0` checks the value *before* decrementing, so `missing` drops only for a character that was genuinely still needed — a fourth `'a'` when only three were required does not count. On the way out, `++need[l] > 0` raises `missing` only when removing that character actually breaks validity.

Store `bestL` and `bestLen` and call `substr` exactly once at the end.

Complexity $O(n + m)$ time, $O(128) = O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    string minWindow(string s, string t) {
        if (t.empty() || s.size() < t.size()) return "";
        int need[128] = {0};
        for (char c : t) need[(unsigned char)c]++;
        int missing = t.size(); // characters still owed
        int l = 0, bestL = 0, bestLen = INT_MAX;
        for (int r = 0; r < (int)s.size(); r++) {
            // post-decrement: only a character that was still needed counts
            if (need[(unsigned char)s[r]]-- > 0) missing--;
            while (missing == 0) { // valid -> record, THEN shrink
                if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
                if (++need[(unsigned char)s[l]] > 0) missing++;
                l++;
            }
        }
        return bestLen == INT_MAX ? "" : s.substr(bestL, bestLen);
    }
};
```

WATCH OUT

Duplicates in `t` are the case that separates working code from nearly-working code. `t = "AABC"` requires two `A`s, and any solution that tracks a set of distinct characters rather than counts will accept a window with only one.

19 LC 49 · Group Anagrams

MEDIUM Canonical signature

PROBLEM

Group the strings in an array so that anagrams of each other end up in the same group. The order of groups and of strings within a group does not matter.

APPROACH

Map every word to a **canonical form** — a value that is identical for all members of its group — and use it as a hash-map key. The sorted word is the obvious choice: "eat", "tea" and "ate" all sort to "aet".

The alternative signature is the 26-slot count rendered as a string, which is $O(k)$ per word instead of $O(k \log k)$. Say both; write the sorted one, because it is four lines shorter and words are usually short.

Complexity $O(n \cdot k \log k)$ time for n words of length k . $O(n \cdot k)$ space.

C++ SOLUTION

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> groups;
        for (const string& w : strs) {
            string key = w;
            sort(key.begin(), key.end()); // canonical form: "eat", "tea" -> "aet"
            groups[key].push_back(w);
        }
        vector<vector<string>> res;
        for (auto& kv : groups) res.push_back(move(kv.second));
        return res;
    }
};
```

WATCH OUT

If you use the count-vector signature, the separators are essential: "#1#0#...", not "10...". Without them, counts of 1 and 12 produce the same digit string as counts of 11 and 2, and unrelated words collide.

20 LC 5 · Longest Palindromic Substring

MEDIUM Expand around centre

PROBLEM

Return the longest palindromic substring of `s`.

APPROACH

A string of length n has exactly $2n - 1$ centres — n sitting on a character (odd-length palindromes) and $n - 1$ sitting between two characters (even-length). Every palindromic substring has exactly one centre, so enumerating centres enumerates palindromes with no duplication.

For each centre, expand outwards while the characters match, and keep the longest span seen.

This beats the $O(n^2)$ -space DP table on space while matching it on time. Manacher's algorithm gets it to $O(n)$; naming it is worth credit, writing it under time pressure usually is not.

Complexity $O(n^2)$ time, $O(1)$ extra space.

C++ SOLUTION

```
class Solution {
public:
    string longestPalindrome(string s) {
        int n = s.size();
        if (n == 0) return "";
        int bestL = 0, bestLen = 1;
        auto expand = [&](int l, int r) {
            while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
            int len = r - l - 1; // both overshoot by one on exit
            if (len > bestLen) { bestLen = len; bestL = l + 1; }
        };
        for (int c = 0; c < n; c++) {
            expand(c, c); // odd-length centre
            expand(c, c + 1); // even-length centre
        }
        return s.substr(bestL, bestLen);
    }
};
```

WATCH OUT

When the expand loop exits, `l` and `r` have each overshoot by one. The length is therefore `r - l - 1` and the start is `l + 1`. Writing `r - l + 1` — correct for an inclusive range, wrong here — is the most common error on this problem.

21 LC 647 · Palindromic Substrings

MEDIUM Expand around centre

PROBLEM

Count every palindromic substring of `s`. Substrings with the same text but different positions count separately.

APPROACH

The same $2n - 1$ centres as LC 5, counting instead of measuring: every successful expansion step *is* one more palindromic substring, so increment a counter inside the expand loop.

Being able to write LC 5 and LC 647 as the same loop with a different body is the point of doing both.

Complexity $O(n^2)$ time, $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int countSubstrings(string s) {
        int n = s.size(), total = 0;
        auto expand = [&](int l, int r) {
            while (l >= 0 && r < n && s[l] == s[r]) { total++; l--; r++; }
        };
        for (int c = 0; c < n; c++) {           // exactly 2n - 1 centres
            expand(c, c);
            expand(c, c + 1);
        }
        return total;
    }
};
```

WATCH OUT

Increment the counter **inside** the `while`, before expanding further — each successful match is its own palindrome. Counting once per centre after the loop counts only the longest one and undercounts badly.

22 LC 680 · Valid Palindrome II

EASY First mismatch, two tries

PROBLEM

Return true if `s` can be made a palindrome by deleting **at most one** character.

APPROACH

You do not need to try deleting each character in turn. Walk two pointers inwards as normal; the **first** mismatch is the only place a deletion could ever help, and there are exactly two candidates — drop the left character or drop the right one.

So on the first mismatch, check whether either of the two remaining ranges is a plain palindrome, and return the OR. If the pointers meet without any mismatch, the string was already a palindrome.

That argument — one decision point, two branches — is what makes this $O(n)$ rather than $O(n^2)$, and it is what the interviewer is listening for.

Complexity $O(n)$ time — the helper runs at most twice. $O(1)$ space.

C++ SOLUTION

```
class Solution {
    bool isPalRange(const string& s, int l, int r) {
        while (l < r) if (s[l++] != s[r--]) return false;
        return true;
    }
public:
    bool validPalindrome(string s) {
        int l = 0, r = (int)s.size() - 1;
        while (l < r) {
            if (s[l] != s[r])
                // the FIRST mismatch is the only place a deletion can help,
                // and there are exactly two candidates
                return isPalRange(s, l + 1, r) || isPalRange(s, l, r - 1);
            l++;
            r--;
        }
        return true;
    }
};
```

WATCH OUT

Take the range-based helper `isPalRange(s, l, r)` rather than building substrings.

`isPal(s.substr(l+1, r-l))` is correct but allocates, and it makes the index arithmetic much easier to get wrong.

23 LC 151 · Reverse Words in a String

MEDIUM Tokenise, or three reversals

PROBLEM

Reverse the order of the words in a string. Words are separated by one or more spaces; leading and trailing spaces must be removed and inner runs collapsed to a single space.

APPROACH

`stringstream`'s `operator>>` handles every whitespace rule for free: it skips arbitrary runs, leading and trailing. Read the words into a vector, then join them in reverse with single spaces.

The $O(1)$ -space follow-up is worth knowing: reverse the whole string, then reverse each word in place, then squeeze out the extra spaces with a read/write pointer. Three passes, no extra container. Offer it after the simple version works.

Complexity $O(n)$ time, $O(n)$ space for the words ($O(1)$ extra for the three-reversal version).

C++ SOLUTION

```
class Solution {
public:
    string reverseWords(string s) {
        // operator>> skips arbitrary runs of whitespace, leading and trailing
        stringstream ss(s);
        vector<string> words;
        string w;
        while (ss >> w) words.push_back(w);
        string res;
        for (int i = (int)words.size() - 1; i >= 0; i--) {
            res += words[i];
            if (i) res += ' ';           // no trailing space
        }
        return res;
    }
};
```

WATCH OUT

The trailing space. Appending `word + ' '` for every word leaves one at the end, and the judge checks for it. Append the separator only when the word is not the last one.

24 LC 443 · String Compression

MEDIUM Read/write pointer

PROBLEM

Compress a `vector<char>` in place: each run of repeated characters becomes the character followed by the run length, with the length omitted when it is 1. Return the new length.

APPROACH

The read/write pattern. `read` scans a whole run, `write` emits the character and — when the run is longer than 1 — its digits.

The invariant that makes this safe is that `write` never overtakes `read`: a run of length k is replaced by at most $1 + \text{digits}(k)$ characters, which is $\leq k$ for every $k \geq 1$. Say that sentence when asked why in-place is legal here.

Complexity $O(n)$ time, $O(1)$ extra space.

C++ SOLUTION

```
class Solution {
public:
    int compress(vector<char>& chars) {
        int n = chars.size(), read = 0, write = 0;
        while (read < n) {
            char c = chars[read];
            int run = 0;
            while (read < n && chars[read] == c) { read++; run++; }
            chars[write++] = c;
            if (run > 1)
                for (char d : to_string(run)) // run can be multi-digit!
                    chars[write++] = d;
        }
        return write; // write never overtakes read
    }
};
```

WATCH OUT

The run length can be **multi-digit**. `chars[write++] = run + '0'` looks right and breaks on any run of 10 or more. Convert with `to_string(run)` and write the digits one at a time.

25 LC 8 · String to Integer (atoi)

MEDIUM Parsing, case enumeration

PROBLEM

Implement `atoi`: skip leading spaces, read an optional sign, read digits until a non-digit, and clamp the result to the 32-bit signed range.

APPROACH

There is no algorithm here — only five steps that must happen in exactly this order: skip spaces, read at most one sign, accumulate digits, clamp, stop at the first non-digit.

Clamp **inside** the digit loop, not after it. A 30-digit input overflows a 64-bit integer long before the loop ends, so the overflow test has to run on every digit. The clean form is `if (res > (INT_MAX - d) / 10)` `return ...` which never overflows at all.

Complexity $O(n)$ time, $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int myAtoi(string s) {
        int i = 0, n = s.size();
        while (i < n && s[i] == ' ') i++;           // 1. leading spaces
        int sign = 1;
        if (i < n && (s[i] == '+' || s[i] == '-'))    // 2. at most one sign
            sign = (s[i++] == '-') ? -1 : 1;
        int res = 0;
        while (i < n && isdigit((unsigned char)s[i])) { // 3. digits
            int d = s[i++] - '0';
            // 4. clamp on EVERY digit - a 30-digit input overflows long long too
            if (res > (INT_MAX - d) / 10)
                return sign == 1 ? INT_MAX : INT_MIN;
            res = res * 10 + d;
        }
        return sign * res;                          // 5. stop at non-digit
    }
};
```

WATCH OUT

The range is asymmetric: `INT_MIN` is `-2147483648` while `INT_MAX` is `2147483647`, so the negative side allows one more. Clamping the magnitude at `INT_MAX` and then negating gets `"-2147483648"` wrong.

26 LC 165 · Compare Version Numbers

MEDIUM Parallel parsing

PROBLEM

Compare two dot-separated version strings. Return -1 , 0 or 1 . Leading zeros in a part are insignificant, and a missing trailing part is treated as zero — so `"1.0"` and `"1.0.0"` are equal.

APPROACH

Parse both strings in lockstep, one part at a time, comparing the parsed integers as you go. Because a missing part parses as 0 , the two rules fall out for free without any padding or splitting.

The loop condition is `while (i < n || j < m)` — **or**, not **and**. Once one string is exhausted, its remaining parts are 0 and the comparison must keep going against the other's.

Complexity $O(n + m)$ time, $O(1)$ space.

C++ SOLUTION

```
class Solution {
public:
    int compareVersion(string version1, string version2) {
        int i = 0, j = 0, n = version1.size(), m = version2.size();
        while (i < n || j < m) { // OR, not AND: a missing part is 0
            long a = 0, b = 0;
            while (i < n && version1[i] != '.') a = a * 10 + (version1[i++] - '0');
            while (j < m && version2[j] != '.') b = b * 10 + (version2[j++] - '0');
            if (a != b) return a < b ? -1 : 1;
            i++; // step over the '.', harmless past the end
            j++;
        }
        return 0;
    }
};
```

WATCH OUT

Comparing the strings lexicographically fails on `"1.2"` versus `"1.10"`, because `'2' > '1'` as characters while $2 < 10$ as numbers. The comparison must be numeric, part by part.

27 LC 6 · Zigzag Conversion

MEDIUM

Row simulation

PROBLEM

Write the string in a zigzag across `numRows` rows, then read it off row by row.

APPROACH

Do not compute geometry. Simulate: keep a current row and a direction that flips whenever you touch the top or bottom row, appending each character to its row's buffer. Concatenate the rows at the end.

Initialising `dir = -1` so that the flip at row 0 makes it `+1` avoids an extra branch before the loop.

Complexity $O(n)$ time, $O(n)$ space for the row buffers.

C++ SOLUTION

```
class Solution {
public:
    string convert(string s, int numRows) {
        if (numRows == 1) return s;           // otherwise cur walks out of range
        vector<string> rows(min<int>(numRows, s.size()));
        int cur = 0, dir = -1;                // first flip at row 0 makes dir = +1
        for (char c : s) {
            rows[cur] += c;
            if (cur == 0 || cur == numRows - 1) dir = -dir;
            cur += dir;
        }
        string res;
        for (const string& r : rows) res += r;
        return res;
    }
};
```

WATCH OUT

`numRows == 1` must be special-cased. With one row, row 0 is also the last row, the direction flips on every character, and the row index walks straight out of range.

28 LC 179 · Largest Number

MEDIUM

Custom comparator `a+b > b+a`

PROBLEM

Arrange the given non-negative integers so that their concatenation is the largest possible number, and return it as a string.

APPROACH

Sort with a comparator that compares the two **concatenations**, not the two strings: `a` comes before `b` when `a + b > b + a`.

That is what makes `"9"` beat `"34"` (`"934" > "349"`) and `"3"` beat `"30"` (`"330" > "303"`). Comparing the strings directly, or comparing the numbers, gets both of those backwards.

This comparator is a genuine strict weak ordering — it behaves like comparing the infinite repetitions of each string — which is what makes `sort` legal rather than undefined behaviour. Interviewers do ask why.

Complexity $O(n \cdot k \log n)$ time for n numbers of k digits. $O(n \cdot k)$ space.

C++ SOLUTION

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> v;
        for (int x : nums) v.push_back(to_string(x));
        // compare the two concatenations, not the two strings:
        // "9" must beat "34", and "3" must beat "30" ("330" > "303")
        sort(v.begin(), v.end(),
            [](const string& a, const string& b) { return a + b > b + a; });
        if (v[0] == "0") return "0"; // all zeros collapse to a single "0"
        string res;
        for (const string& s : v) res += s;
        return res;
    }
};
```

WATCH OUT

All-zero input. `{0, 0}` concatenates to `"00"`, and the expected answer is `"0"`. One guard — if the first element after sorting is `"0"`, return `"0"` — handles every such case.

Tier 3 · Advanced

KMP, hashing and the two-string DP grid. These are what separate a good on-campus result from a great one — and the prefix function repays memorising outright.

29 LC 214 · Shortest Palindrome

HARD KMP on `s + '#' + rev(s)`

PROBLEM

Prepend the fewest possible characters to `s` to make it a palindrome, and return the result.

APPROACH

The characters you prepend are forced: whatever follows the **longest palindromic prefix** of `s` has to be mirrored onto the front. So the problem reduces to finding that prefix.

Now the transformation. The longest palindromic prefix of `s` is the longest prefix of `s` that is also a suffix of `reverse(s)` — which is exactly what the prefix function computes on the concatenation `s + '#' + reverse(s)`. Its last entry is the answer's length.

That is the whole solution: one prefix function, one `substr`, one concatenation.

Complexity $O(n)$ time, $O(n)$ space.

C++ SOLUTION

```
class Solution {
    vector<int> prefixFunction(const string& p) {
        int m = p.size();
        vector<int> pi(m, 0);
        for (int i = 1; i < m; i++) {
            int j = pi[i - 1];
            while (j > 0 && p[i] != p[j]) j = pi[j - 1]; // fall back to the
            if (p[i] == p[j]) j++; // next-longest border
            pi[i] = j;
        }
        return pi;
    }
public:
    string shortestPalindrome(string s) {
        if (s.empty()) return s;
        string rev(s.rbegin(), s.rend());
        // '#' must not occur in s, or a border can straddle the join
        string comb = s + '#' + rev;
        vector<int> pi = prefixFunction(comb);
        int k = pi.back(); // longest palindromic PREFIX of s
        return rev.substr(0, s.size() - k) + s;
    }
};
```

WATCH OUT

The `'#'` separator is load-bearing. Without a character that cannot appear in `s`, the border can straddle the join and report a length greater than `s.size()`, producing garbage on inputs like `"aaaa"`.

30 LC 459 · Repeated Substring Pattern

EASY Prefix function, period test

PROBLEM

Return true if `s` can be built by taking some proper substring of it and repeating it two or more times.

APPROACH

`n - pi[n-1]` is the **smallest period** of the string — the shortest block that, repeated, generates it (possibly with a partial copy at the end). The string is a whole number of repetitions exactly when that period divides `n` evenly and the border is non-empty.

Two conditions, one line: `pi[n-1] > 0 && n % (n - pi[n-1]) == 0`.

The well-known one-liner `(s + s).substr(1, 2n - 2).find(s) != npos` is correct and cute, and it is $O(n^2)$ with `find`. Mention it, write the prefix-function version.

Complexity $O(n)$ time, $O(n)$ space for the prefix array.

C++ SOLUTION

```
class Solution {
    vector<int> prefixFunction(const string& p) {
        int m = p.size();
        vector<int> pi(m, 0);
        for (int i = 1; i < m; i++) {
            int j = pi[i - 1];
            while (j > 0 && p[i] != p[j]) j = pi[j - 1];
            if (p[i] == p[j]) j++;
            pi[i] = j;
        }
        return pi;
    }
public:
    bool repeatedSubstringPattern(string s) {
        int n = s.size();
        vector<int> pi = prefixFunction(s);
        int period = n - pi[n - 1]; // the smallest period of s
        return pi[n - 1] > 0 && n % period == 0;
    }
};
```

WATCH OUT

The `pi[n-1] > 0` guard is not decoration. Without it, a string with no border at all gives a period equal to `n`, which divides `n`, and every string returns true.

31 LC 1392 · Longest Happy Prefix

HARD Prefix function, directly

PROBLEM

Return the longest prefix of `s` that is also a suffix of `s`, excluding `s` itself.

APPROACH

This problem is the definition of the prefix function's last entry. Build `pi` and return `s.substr(0, pi[n-1])`.

It is labelled Hard only because it assumes you know KMP. If you can write `prefixFunction` from memory, this is a three-line solution — which is the best possible argument for memorising those twelve lines outright.

Complexity $O(n)$ time, $O(n)$ space.

C++ SOLUTION

```
class Solution {
public:
    string longestPrefix(string s) {
        int n = s.size();
        vector<int> pi(n, 0);
        for (int i = 1; i < n; i++) {
            int j = pi[i - 1];
            while (j > 0 && s[i] != s[j]) j = pi[j - 1];
            if (s[i] == s[j]) j++;
            pi[i] = j;
        }
        // the answer IS the definition of the prefix function's last entry:
        // the longest proper prefix that is also a suffix
        return s.substr(0, pi[n - 1]);
    }
};
```

WATCH OUT

The word **proper** matters: the whole string is trivially both a prefix and a suffix of itself and does not count. The prefix function already excludes it by construction, which is why this works without any extra check.

32 LC 187 · Repeated DNA Sequences

MEDIUM Fixed window + rolling bitmask

PROBLEM

Find every 10-letter-long substring that occurs more than once in a DNA string over the alphabet **A**, **C**, **G**, **T**.

APPROACH

Four letters need 2 bits each, so a 10-letter window packs into exactly **20 bits**. Roll it forward with a shift, an OR and a mask — $O(1)$ per step — and this is not a hash at all but a perfect encoding, so there are no collisions to verify.

Count each code in a hash map and emit the substring the moment its count reaches 2. Testing `== 2` rather than `>= 2` is what keeps duplicates out of the output without a second pass.

Complexity $O(n)$ time, $O(n)$ space for the map.

C++ SOLUTION

```
class Solution {
public:
    vector<string> findRepeatedDnaSequences(string s) {
        int n = s.size();
        vector<string> res;
        if (n < 10) return res;
        int code[128] = {0};
        code['A'] = 0; code['C'] = 1; code['G'] = 2; code['T'] = 3;
        // 10 characters over a 4-letter alphabet pack into 20 bits exactly,
        // so this "hash" is a perfect encoding - no collisions to verify
        int mask = (1 << 20) - 1, h = 0;
        unordered_map<int,int> seen;
        for (int i = 0; i < n; i++) {
            h = ((h << 2) | code[(unsigned char)s[i]]) & mask; // roll in  $O(1)$ 
            if (i >= 9 && ++seen[h] == 2) res.push_back(s.substr(i - 9, 10));
        }
        return res;
    }
};
```

WATCH OUT

A generic polynomial rolling hash works too, but then a hash match is only a *candidate* and you must verify it with a real string comparison. The 2-bit packing sidesteps that entirely — say why when you choose it.

33 LC 516 · Longest Palindromic Subsequence

MEDIUM Interval DP

PROBLEM

Find the length of the longest palindromic **subsequence** of `s` — characters keep their order but gaps are allowed.

APPROACH

`dp[i][j]` is the answer for the range `s[i..j]`. If the two ends match they can both be used, giving `dp[i+1][j-1] + 2`. If they do not, one of them must be discarded, so take the better of `dp[i+1][j]` and `dp[i][j-1]`.

There is an elegant alternative: **LPS(s) = LCS(s, reverse(s))**. It is a one-liner given the LCS routine from problem 34, and it is a satisfying thing to point out even if you then write the direct version.

Complexity $O(n^2)$ time, $O(n^2)$ space (reducible to $O(n)$).

C++ SOLUTION

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n, 0));
        // dp[i][j] depends on dp[i+1][j-1], so i must DESCEND and j ascend
        for (int i = n - 1; i >= 0; i--) {
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++)
                dp[i][j] = (s[i] == s[j]) ? dp[i + 1][j - 1] + 2
                    : max(dp[i + 1][j], dp[i][j - 1]);
        }
        return dp[0][n - 1];
    }
};
```

WATCH OUT

`dp[i][j]` depends on `dp[i+1][j-1]` — a larger `i`. So `i` must **descend** and `j` ascend. A plain row-major fill reads uninitialised cells and returns plausible-looking nonsense.

34 LC 1143 · Longest Common Subsequence

MEDIUM Two-string DP grid

PROBLEM

Return the length of the longest subsequence common to both strings.

APPROACH

The canonical two-string grid. `dp[i][j]` is the answer for the first `i` characters of one string and the first `j` of the other — `i` and `j` are **counts**, not indices, which is what makes `dp[0][j]` a meaningful base case.

If the last characters match they can be paired: take the diagonal plus one. If not, one of them has to be dropped: try both and keep the better.

LCS is the parent of a whole family. Minimum deletions to make two strings equal is `n + m - 2 · LCS`; the shortest common supersequence is `n + m - LCS`; the longest common *substring* is the same grid with the mismatch case resetting to 0 and the answer taken from the maximum cell rather than the corner.

Complexity $O(n \cdot m)$ time, $O(n \cdot m)$ space — reducible to $O(m)$ with two rolling rows.

C++ SOLUTION

```
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int n = text1.size(), m = text2.size();
        // dp[i][j] = LCS of the first i chars of text1 and first j of text2
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                // i and j are COUNTS, so the characters are at i-1 and j-1
                dp[i][j] = (text1[i - 1] == text2[j - 1])
                    ? dp[i - 1][j - 1] + 1
                    : max(dp[i - 1][j], dp[i][j - 1]);
        return dp[n][m];
    }
};
```

WATCH OUT

Because `i` and `j` are counts, the characters being compared are `a[i-1]` and `b[j-1]`. Writing `a[i] == b[j]` reads one character too far and silently shifts the whole table.

35 LC 72 · Edit Distance

MEDIUM Two-string DP grid

PROBLEM

Find the minimum number of single-character insertions, deletions or replacements needed to turn `word1` into `word2`.

APPROACH

The same grid as LCS with a different recurrence. When the characters match there is nothing to pay — take the diagonal. When they differ, pay 1 and take the best of three neighbours.

Learn the three by what they mean, not by position: the diagonal is *replace*, the cell above is *delete from word1*, the cell to the left is *insert into word1*.

The base cases are the part people skip: `dp[i][0] = i` because turning an *i*-character string into the empty string costs *i* deletions, and `dp[0][j] = j` symmetrically.

Complexity $O(n \cdot m)$ time, $O(n \cdot m)$ space — reducible to $O(m)$.

C++ SOLUTION

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int n = word1.size(), m = word2.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
        for (int i = 0; i <= n; i++) dp[i][0] = i; // delete everything
        for (int j = 0; j <= m; j++) dp[0][j] = j; // insert everything
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                dp[i][j] = (word1[i - 1] == word2[j - 1])
                    ? dp[i - 1][j - 1]
                    : 1 + min({dp[i - 1][j - 1], // replace
                             dp[i - 1][j],      // delete from word1
                             dp[i][j - 1]});    // insert into word1
        return dp[n][m];
    }
};
```

WATCH OUT

Zero-initialising the table and omitting the two base-case loops produces an answer that is correct on most sample inputs and wrong whenever one string is a prefix of the other. Always write those two loops first.

36 LC 10 · Regular Expression Matching

HARD DP, `*` binds to `p[j-2]`

PROBLEM

Implement matching for `.` (any single character) and `*` (zero or more of the **preceding** element), where the match must cover the entire input string.

APPROACH

`*` is never standalone — it always binds to the character before it, so `p[j-1] == '*'` must be read together with `p[j-2]`.

Two cases. Zero occurrences: skip both pattern characters, `dp[i][j] = dp[i][j-2]`. One more occurrence: only if `p[j-2]` matches `s[i-1]`, in which case `dp[i-1][j]` — consume a text character and keep the `*` available.

The base row matters: patterns like `"abc*"` match the empty string, which is what the `j` loop starting at 2 sets up.

Complexity $O(n \cdot m)$ time, $O(n \cdot m)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool isMatch(string s, string p) {
        int n = s.size(), m = p.size();
        vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
        dp[0][0] = true;
        for (int j = 2; j <= m; j++) // "a*b*c*" can match empty
            if (p[j - 1] == '*') dp[0][j] = dp[0][j - 2];
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++) {
                if (p[j - 1] == '*') {
                    // '*' binds to p[j-2], so zero occurrences skips TWO
                    dp[i][j] = dp[i][j - 2];
                    if (p[j - 2] == '.' || p[j - 2] == s[i - 1])
                        dp[i][j] = dp[i][j] || dp[i - 1][j]; // one more of it
                } else {
                    dp[i][j] = dp[i - 1][j - 1] &&
                        (p[j - 1] == '.' || p[j - 1] == s[i - 1]);
                }
            }
        return dp[n][m];
    }
};
```

WATCH OUT

The zero-occurrence case skips **two** pattern characters, not one. `dp[i][j-1]` is the wildcard rule from LC 44 and is wrong here — the two problems look nearly identical and this is the line that differs.

37 LC 44 · Wildcard Matching

HARD DP, * standalone

PROBLEM

Implement matching for `?` (exactly one character) and `*` (any sequence, including empty), covering the entire input string.

APPROACH

Here `i` is independent of what precedes it, which makes the recurrence simpler than LC 10's. Two options joined by an OR: `dp[i-1][j]` — the `i` swallows `s[i-1]` and remains available — or `dp[i][j-1]` — the `*` matches nothing and we move past it.

The base row is the run of leading `i`'s: `dp[0][j]` is true only while every pattern character so far has been a `i`.

There is also a well-known greedy two-pointer solution in $O(1)$ space that backtracks to the last `*`. It is faster but much harder to defend under pressure; write the DP and mention the greedy.

Complexity $O(n \cdot m)$ time, $O(n \cdot m)$ space.

C++ SOLUTION

```
class Solution {
public:
    bool isMatch(string s, string p) {
        int n = s.size(), m = p.size();
        vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
        dp[0][0] = true;
        for (int j = 1; j <= m; j++) // only leading '*'s match empty
            dp[0][j] = dp[0][j - 1] && p[j - 1] == '*';
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++) {
                if (p[j - 1] == '*')
                    // here '*' stands alone: swallow s[i-1], or match nothing
                    dp[i][j] = dp[i - 1][j] || dp[i][j - 1];
                else
                    dp[i][j] = dp[i - 1][j - 1] &&
                        (p[j - 1] == '?' || p[j - 1] == s[i - 1]);
            }
        return dp[n][m];
    }
};
```

WATCH OUT

Stating the difference from LC 10 crisply is half the interview: in LC 44, `i` is standalone and consumes text via `dp[i-1][j]`; in LC 10, `i` is glued to `p[j-2]` and zero-occurrences skips two via `dp[i][j-2]`.

38 LC 97 · Interleaving String

MEDIUM DP, index `c[i+j-1]`

PROBLEM

Return true if `s3` is formed by interleaving `s1` and `s2`, preserving the relative order within each.

APPROACH

`dp[i][j]` asks whether the first `i` characters of `s1` and the first `j` of `s2` can interleave to form the first `i + j` of `s3`. The cell is reachable if either the previous cell up supplies `s1[i-1]` or the previous cell left supplies `s2[j-1]`, matching `s3[i+j-1]`.

That index is the whole problem: having consumed `i + j` characters, the one being placed is number `i + j` of `s3`, which lives at index `i + j - 1`.

Greedy fails, and the reason is worth stating: when `s1[i-1]` and `s2[j-1]` are both equal to the next character of `s3`, there is no local information that says which to take.

Complexity $O(n \cdot m)$ time, $O(n \cdot m)$ space — reducible to $O(m)$.

C++ SOLUTION

```
class Solution {
public:
    bool isInterleave(string s1, string s2, string s3) {
        int n = s1.size(), m = s2.size();
        if (n + m != (int)s3.size()) return false; // the guard that matters
        vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
        dp[0][0] = true;
        for (int i = 0; i <= n; i++)
            for (int j = 0; j <= m; j++) {
                // having consumed i + j characters, the next one is s3[i+j-1]
                if (i && dp[i - 1][j] && s1[i - 1] == s3[i + j - 1]) dp[i][j] = true;
                if (j && dp[i][j - 1] && s2[j - 1] == s3[i + j - 1]) dp[i][j] = true;
            }
        return dp[n][m];
    }
};
```

WATCH OUT

The length guard `n + m == s3.size()` must come first. Without it the table is filled against a string of the wrong length and the index `i + j - 1` reads out of bounds.